# Testing

**What is testing..?**

**Testing = verifying your code behaves correctly under different conditions**

**That's it.**

**🧱 2. Main Types of Testing (Real Classification)**

**Forget confusion — there are 3 levels that matter for you:**

**🔹 1. Manual Testing (Human-driven)**

**👉 You test things yourself**

**Examples:**

- **Calling API in Postman**
- **Logging in browser console**
- **Clicking buttons in UI**

**Tools:**

- **Postman**
- **Browser DevTools (console)**

**Example:**

**POST /book**
**→ check response**
**→ check DB manually**

**✅ Pros:**

- **Fast to start**
- **Good for debugging**

**❌ Cons:**

- **Not scalable**

- You forget to test edge cases
- Not repeatable

🔹 2. Automated Testing (Code-driven)

👉 You write code that tests your code

Tools:

- Jest
- Supertest
- Cypress

Example:

```
expect(res.statusCode).toBe(200);
```

✅ Pros:

- Repeatable
- Reliable
- Used in companies

❌ Cons:

- Takes time to write
- Requires discipline

🔹 3. Exploratory / Debug Testing (Dev-style)

👉 This is what you're doing most right now

Examples:

- console.log()
- Checking Redis manually
- Watching logs

This is:

- **NOT formal testing**
- **BUT very important while building**

## Why we need testing .?

**Not to prove your code works *now* —**
**but to ensure it keeps working later.**

**2. Reality of Backend Systems (your case)**

**You're building:**

- **booking system**
- **Redis**
- **async workers**

**That means:**

👉 **Multiple moving parts**
👉 **Race conditions**
👉 **Side effects**

**One small change = system breaks silently**

💥 **3. Without Testing (what actually happens)**

**You:**

- **Fix one bug**
- **Accidentally break something else**
- **Don't notice**

**Example:**

**Before:**
**tickets = 10 → booking works**

**You change logic…**

**Now:**
**tickets = -1 (overselling)**

**You won't catch this with just Postman unless you re-test everything manually (you won't).**

## 🔥 4. What Testing Actually Gives You

### ✅ 1. Confidence

**You can change code without fear**

---

### ✅ 2. Regression Protection

**Old features don't break when you add new ones**

---

### ✅ 3. Edge Case Safety

**You force yourself to think:**

**What if:**
**- 2 users book at same time?**
**- tickets = 0?**
**- Redis fails?**

---

### ✅ 4. Documentation (hidden benefit)

**Tests show how your system is supposed to behave**

## Think Like a Real Backend Engineer

**Amateurs think:**

**"Does it work?"**

**Professionals think:**

**"Can it break?"**

**“Testing ensures code reliability by preventing regressions, validating edge cases, and giving confidence to make changes without breaking existing functionality.”**

**we write test cases to validate expected behavior. If future changes break that behavior, tests fail and alert us. However, tests are only as good as their coverage — they must include edge cases and critical flows to be effective.**

## JEST

JEST is the most widely used framework for MERN testing

### 🚀 Step 1: Install Jest

In your backend project:

npm install --save-dev jest /

FOR TS

npm install --save-dev jest ts-jest @types/jest typescript  for st

npx ts-jest config:init

This creates:

jest.config.js

### ⚙️ Step 2: Setup package.json

```
{
  "scripts": {
    "test": "jest"
  }
}
```

Now you can run:

npm test

### 🧪 Step 3: Your First Test (VERY IMPORTANT)

Create file:

sum.test.js

```
function sum(a, b) {
  return a + b;
}

test("adds 2 + 3 = 5", () => {
  expect(sum(2, 3)).toBe(5);
});
```

Run:

```
npm test
```

👉 If it passes → setup is correct

---

### 🧠 Step 4: Understand Core Syntax (this is all you need)

```
test("description", () => {
  expect(actual).toBe(expected);
});
```

That’s 80% of Jest.

### 🧠 What describe actually does (in Jest)

describe is just for:

**Grouping related tests together**

---

#### 🔹 Example WITHOUT describe

```
test("add works", () => {
  expect(2 + 2).toBe(4);
});

test("subtract works", () => {
```

```
  expect(5 - 2).toBe(3);
});
```

---

🔹 **Example WITH describe**

```
describe("Math operations", () => {
  test("add works", () => {
    expect(2 + 2).toBe(4);
  });

  test("subtract works", () => {
    expect(5 - 2).toBe(3);
  });
});
```

---

⚔️ **Difference?**

✅ **What changes:**

- Better **organization**
- Better **readable test output**

Example output:

```
Math operations
  ✓ add works
  ✓ subtract works
```

Now testing with Express app

🚀 **Step 1: Install**

npm install --save-dev jest supertest

🧠 **What is Supertest?**

👉 **Supertest** is a library that lets you:

**send HTTP requests to your Express app and test the response — without running a real server**

---

⚡ **In simple terms**

Instead of:

- Opening Postman
- Hitting localhost:3000 manually

👉 Supertest does it **inside your code**

Do all the Jest setup

⚙️ **Step 2: Export your Express app (IMPORTANT)**

Most beginners mess this up.

❌ **Don't do this:**

```
app.listen(3000);
```

🧠 **Why NOT use app.listen() in tests?**

👉 Because it **starts a real server on a port**

Problems:

- ❌ Port conflicts (3000 already used)
- ❌ Slower tests
- ❌ Hard to control/start/stop
- ❌ Not isolated

---

⚡ **How Supertest handles it**

👉 Supertest **does NOT start a real server**

Instead:

- It directly hooks into your Express app
- Uses Node's internal HTTP handling

request(app) // not localhost:3000

👉 It simulates requests **in memory**

🔥 **Mental Model**

app.listen() → real network server (port-based)

Supertest → fake request → directly into app → gets response

"We avoid app.listen() in tests because it starts a real server and causes port conflicts. Supertest directly sends requests to the Express app in memory without binding to a port."

---

✅ **Do this:**

```
// app.js
import express from "express";

const app = express();

app.use(express.json());
```

```
app.get("/test", (req, res) => {
  res.json({ message: "ok" });
});

export default app;
```

**Separate server file:**

```
// server.js
import app from "./app.js";

app.listen(3000, () => {
  console.log("Server running");
});
```

👉 This separation is **critical for testing**

### 🧪 Step 3: First API Test

```
// test/app.test.js
import request from "supertest";
import app from "../app.js";

describe("GET /test", () => {
  test("should return ok message", async () => {
    const res = await request(app).get("/test");

    expect(res.statusCode).toBe(200);
    expect(res.body.message).toBe("ok");
  });
});
```

### ▶️ Step 4: Run

```
npm test
```

👉 You just tested an API like a real backend dev

Request from Supertest

```
request(app)
```

send your app instance

Then you chain methods:

```
.get() | .post() | .put() | .delete()
.send()
.set()
.query()
```

### 🚀 1. Sending Query Params (?page=1)

**Route:**

```
app.get("/users", (req, res) => {
  res.json({ page: req.query.page });
});
```

**Test:**

```
const res = await request(app)
  .get("/users")
  .query({ page: 1 });

expect(res.body.page).toBe("1");
```

### 🚀 2. Sending Route Params (/user/:id)

👉 Just include in URL

```
const res = await request(app)
  .get("/user/123");

expect(res.statusCode).toBe(200);
```

## 🚀 3. Sending Body (POST / PUT)

**Route:**

```
app.post("/book", (req, res) => {
  res.json({ tickets: req.body.tickets });
});
```

**Test:**

```
const res = await request(app)
  .post("/book")
  .send({ tickets: 5 });

expect(res.body.tickets).toBe(5);
```

👉 .send() = body

## 🚀 4. Sending Headers (Auth, etc.)

```
const res = await request(app)
  .get("/profile")
  .set("Authorization", "Bearer token123");

expect(res.statusCode).toBe(200);
```

## 🚀 5. Combining Everything (REAL CASE)

```
const res = await request(app)
  .post("/events/123/book")
  .set("Authorization", "Bearer token123")
  .query({ fast: true })
  .send({ tickets: 2 });

expect(res.statusCode).toBe(200);
```

## ⚔️ Full Cheat Sheet

.get("/url")  → GET request
.post("/url")  → POST request
.send({})  → body
.query({})  → query params
.set("key", "val")  → headers

Types of testing

🧠 **1. Big Picture First**

All testing types differ by:

👉 **How much of the system you are testing**

Unit → small piece
Integration → multiple pieces together
E2E → full system (real user flow)

🔹 **2. Unit Testing (smallest level)**

👉 Test **single function / logic in isolation**

**Example:**

```
function bookTicket(tickets) {
  return tickets - 1;
}
```

Test:

```
test("reduce ticket", () => {
  expect(bookTicket(5)).toBe(4);
});
```

---

🧠 **Key idea:**

- No DB ❌
- No API ❌
- No Redis ❌

👉 Only logic

---

✅ **Use when:**

- Testing business logic
- Fast checks

---

🔹 **3. Integration Testing (most important for you)**

👉 Test **multiple parts working together**

**Example (your case):**

API → Controller → DB → Response

Test:

```
await request(app)
  .post("/book")
  .send({ tickets: 5 });
```

---

🧠 **Key idea:**

- API + DB + logic together
- Real flow

---

✅ **Includes:**

- Express routes
- MongoDB
- Middleware
- Sometimes Redis

---

👉 This is what you are doing with:

- Jest
- Supertest

---

## 🔹 4. End-to-End Testing (E2E)

👉 Test **entire system like a real user**

**Example:**

User clicks → frontend → API → DB → response → UI update

---

**Tools:**

- Cypress
- Playwright

---

**Example:**

- Open website
- Click “Book Ticket”
- See success message

---

🧠 **Key idea:**

- Includes frontend + backend
- Real user simulation

---

## ⚔️ 5. Clean Comparison (THIS IS GOLD)

| Type | Scope | Speed | Used for |
|---|---|---|---|
| Unit | Single function | ⚡ Fast | Logic |
| Integration | Multiple layers | ⚡ Medium | APIs |
| E2E | Full system | 🐢 Slow | User flow |

---

## 🧠 7. In YOUR Booking Project

**Unit:**

bookTicket() logic

**Integration:**

POST /book → DB update → response

**E2E:**

Frontend click → booking success

---

## 🎯 Interview One-liner

“Unit testing tests individual functions, integration testing verifies multiple components working together like APIs and databases, and end-to-end testing validates the complete system flow from user interaction to final output.”

---

## What is Vitest?

👉 **Vitest**

A fast testing framework built for modern JS (Vite ecosystem)

---

⚔️ **Why Vitest over Jest?**

✅ **1. Much faster (real reason)**

- Uses native ES modules
- Runs tests in parallel efficiently

---

✅ **2. Better for modern projects**

- Works seamlessly with Vite
- Cleaner config

---

✅ **3. Jest-like syntax**

test(), expect(), describe()

👉 Almost same as Jest → easy switch

Now setup with express

🚀 **0. Create Project**

```
mkdir ts-vitest-express
cd ts-vitest-express
npm init -y
```

---

### 📦 1. Install Dependencies

```
npm install express
npm install -D typescript vitest supertest ts-node @types/node
@types/express @types/supertest
```

---

### ⚙️ 2. package.json (IMPORTANT)

Replace scripts:

```
{
 "type": "module",
 "scripts": {
  "dev": "node --loader ts-node/esm src/server.ts",
  "test": "vitest"
 }
}
```

---

### 🧱 3. tsconfig.json

```
npx tsc --init
```

Then edit:

```
{
 "compilerOptions": {
  "target": "ESNext",
  "module": "ESNext",
  "moduleResolution": "Node",
  "strict": true,
  "esModuleInterop": true,
  "skipLibCheck": true
 }
}
```

---

### 📁 4. Folder Structure

```
src/
  app.ts
  server.ts
  test/
    app.test.ts
```

---

### 🧠 5. Express App

```ts
// src/app.ts
import express, { Request, Response } from "express";

const app = express();

app.use(express.json());

app.get("/test", (req: Request, res: Response) => {
  res.json({ message: "ok" });
});

app.post("/book", (req: Request, res: Response) => {
  const { tickets }: { tickets: number } = req.body;

  if (tickets <= 0) {
    return res.status(400).json({ error: "No tickets left" });
  }

  res.json({ remaining: tickets - 1 });
});

export default app;
```

---

### 🚀 6. Server (separate)

```ts
// src/server.ts
import app from "./app.js";

app.listen(3000, () => {
```

```
  console.log("Server running on 3000");
});
```

---

### 🧪 7. Test File

```
// test/app.test.ts
import { describe, test, expect } from "vitest";
import request from "supertest";
import app from "../src/app.js";

describe("API Tests", () => {
  test("GET /test", async () => {
    const res = await request(app).get("/test");

    expect(res.statusCode).toBe(200);
    expect(res.body.message).toBe("ok");
  });

  test("POST /book success", async () => {
    const res = await request(app)
      .post("/book")
      .send({ tickets: 5 });

    expect(res.statusCode).toBe(200);
    expect(res.body.remaining).toBe(4);
  });

  test("POST /book fail", async () => {
    const res = await request(app)
      .post("/book")
      .send({ tickets: 0 });

    expect(res.statusCode).toBe(400);
  });
});
```

---

### ▶️ 8. Run Tests

```
npm test
```

---

### ⚠️ 2 Common Errors (you WILL face these)

### ❌ Import error

Fix:

```
import app from "../src/app.js";
```

👉 ESM requires .js extension

## Mocking

👉 **Mocking = replacing a real dependency with a fake version during testing**

---

⚡ **Simple Definition**

“Mocking is creating a fake implementation of a function/module so you can control its behavior in tests.”

---

🔥 **Why we need mocking**

Because in real apps, your code depends on:

- DB (MongoDB)
- APIs
- Redis
- external services

👉 You don’t want to call them in unit tests

💥 **Example in your backend**

**Without mocking**

Controller → Service → DB

---

**With mocking**

Controller → Mocked Service (fake)

👉 DB is removed ❌

---

🔑 **Key Benefits**

✅ **1. Speed**

No DB calls

---

### ✅ 2. Isolation

Test only what you want

---

### ✅ 3. Control

You decide output

return success
return error
return edge case

---

### ⚠️ Important Limitation

Mocking can lie.

👉 Your test may pass
👉 But real system may fail

### 🧠 So remember

Mocking = controlled fake world
Real DB = real world

Mocking with mongoDb

### 🚀 0. Create Project

```
mkdir ts-mongo-mock
cd ts-mongo-mock
npm init -y
```

### 📦 1. Install Dependencies

```
npm install express mongoose
npm install -D typescript vitest supertest ts-node @types/node
@types/express @types/supertest
```

### ⚙️ 2. package.json

```
{
 "type": "module",
 "scripts": {
  "dev": "node --loader ts-node/esm src/server.ts",
  "test": "vitest"
 }
}
```

### 🧱 3. tsconfig.json

```
npx tsc --init
```

Edit:

```
{
 "compilerOptions": {
  "target": "ESNext",
  "module": "ESNext",
  "moduleResolution": "Node",
  "strict": true,
  "esModuleInterop": true
```

```
  }
}
```

### 🧱 5. Mongo Model

```
// src/models/user.model.ts
import mongoose from "mongoose";

const userSchema = new mongoose.Schema({
  name: String,
});

export const User = mongoose.model("User", userSchema);
```

### 🧱 6. Service Layer (IMPORTANT)

```
// src/services/user.service.ts
import { User } from "../models/user.model.js";

export const getUserById = async (id: string) => {
  return await User.findById(id);
};
```

### 🧱 7. Route

```
// src/routes/user.route.ts
import express from "express";
import { getUserById } from "../services/user.service.js";

const router = express.Router();

router.get("/:id", async (req, res) => {
  const user = await getUserById(req.params.id);

  if (!user) return res.status(404).json({ error: "Not found" });
```

```
  res.json(user);
});

export default router;
```

### 🧱 8. App

```
// src/app.ts
import express from "express";
import userRoutes from "./routes/user.route.js";

const app = express();

app.use("/users", userRoutes);

export default app;
```

### 🧪 9. NOW — Mock MongoDB (Core Part)

👉 We will mock **Mongoose model**

---

**Test File**

```
// test/user.test.ts
import { describe, test, expect, vi } from "vitest";
import request from "supertest";
import app from "../src/app.js";

// 🔥 MOCK MongoDB Model
vi.mock("../src/models/user.model", () => ({
  User: {
    findById: vi.fn(),
  },
}));

// import AFTER mock
```

```
import { User } from "../src/models/user.model";

describe("GET /users/:id", () => {
  test("should return user", async () => {
    (User.findById as any).mockResolvedValue({
      _id: "123",
      name: "Prajwal",
    });

    const res = await request(app).get("/users/123");

    expect(res.statusCode).toBe(200);
    expect(res.body.name).toBe("Prajwal");
  });

  test("should return 404 if not found", async () => {
    (User.findById as any).mockResolvedValue(null);

    const res = await request(app).get("/users/123");

    expect(res.statusCode).toBe(404);
  });
});
```

🧠 **What just happened (VERY IMPORTANT)**

Real MongoDB ❌ not used
User.findById → mocked function

So:

API → Service → Mocked DB

---

**Explaination**

**vi → mocking utility (VERY important)**

🔥 MOCK SECTION (Core Part)

```
vi.mock("../src/models/user.model", () => ({
User: {
findById: vi.fn(),
},
}));
```

What it does:

Replace entire module:
../src/models/user.model

With:

Fake version:
User.findById = empty mock function

So now:

Real MongoDB ❌ gone
Fake function ✅ used

⚠️ Critical Rule

```
// import AFTER mock
import { User } from "../src/models/user.model";
```

👉 This must come after vi.mock

Why?

Vitest intercepts import
→ gives mocked version instead of real file

If you import before mocking:
❌ real DB will be used

⚔️ Why this is powerful

- No real DB needed ✅
- Fast tests ⚡
- Full control 🎯

💥 **Critical Insight**

If you DIDN’T mock:

👉 Your test would:

- need DB connection
- be slow
- be flaky

Mock Prisma v5 with vitest

🧱 1. Project Setup

mkdir prisma-app

cd prisma-app

npm init -y

npm install express @prisma/client @prisma/adapter-pg pg

npm install -D typescript ts-node-dev prisma @types/express dotenv vitest supertest @types/supertest

**. package.json**

```
{
 "type": "module",
 "scripts": {
  "dev": "node --loader ts-node/esm src/server.ts",
  "test": "vitest"
 }
}
```

⚙️ **2. TypeScript Setup**

npx tsc --init

tsconfig.json

```
{
 "compilerOptions": {
  "target": "ES2020",
  "module": "CommonJS",
  "rootDir": "./src",
  "outDir": "./dist",
  "strict": true,
```

```
    "esModuleInterop": true
  }
}
```

### 🔌 3. Prisma Init (Latest)

```
npx prisma init
```

---

### 🔐 4. Environment Variable

.env

```
DATABASE_URL="postgresql://user:password@localhost:5432/db"
```

### ⚙️ 5. Prisma Config

prisma.config.ts

```
import "dotenv/config";
import { defineConfig } from "prisma/config";

export default defineConfig({
  schema: "prisma/schema.prisma",
  datasource: {
    url: process.env.DATABASE_URL,
  },
});
```

### 📜 6. Schema

prisma/schema.prisma

```
generator client {
  provider = "prisma-client-js"
  output   = "../src/generated/prisma"
}
```

```
datasource db {
  provider = "postgresql"
}

model User {
  id    String @id @default(uuid())
  name  String
  email String @unique
}
```

### 🛠️ 7. Migrate + Generate

```
Npx prisma generate
```

### 🔗 8. Prisma Client (Adapter)

src/prisma/client.ts

```
import "dotenv/config";
import { PrismaPg } from "@prisma/adapter-pg";
import { PrismaClient } from "../generated/prisma";

const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL!,
});

export const prisma = new PrismaClient({ adapter });
```

### 🧠 9. Service

src/services/user.service.ts

```
import { prisma } from "../prisma/client";

export const getUserById = async (id: string) => {
  return prisma.user.findUnique({
```

```
    where: { id },
  });
};
```

### 🎯 10. Controller

```
export const getUser = async (req, res) => {
  const user = await getUserById(req.params.id);

  if (!user) return res.status(404).json({ message: "User not found" });

  res.json(user);
};
```

---

### 🛣️ 11. Route

```
router.get("/:id", getUser);
```

### 🧪 12. Testing Setup (Vitest + Supertest)

---

### ⚠️ RULES (IMPORTANT)

- Mock Prisma (NOT DB)
- Mock BEFORE imports
- Do NOT touch adapter in tests

src/tests/user.test.ts

```
import { vi } from "vitest";


// ☑ mock FIRST
vi.mock("../prisma/client", () => ({
  prisma: {
    user: {
```

```
    findUnique: vi.fn(),
  },
 },
}));

import request from "supertest";
import { describe, it, expect } from "vitest";
import app from "../app";
import { prisma } from "../prisma/client";

describe("GET /users/:id", () => {
 it("should return user", async () => {
  (prisma.user.findUnique as any).mockResolvedValue({
   id: "123",
   name: "Prajwal",
   email: "test@test.com",
  });

  const res = await request(app).get("/users/123");

  expect(res.status).toBe(200);
  expect(res.body.name).toBe("Prajwal");
 });

 it("should return 404 if not found", async () => {
  (prisma.user.findUnique as any).mockResolvedValue(null);
```

```
    const res = await request(app).get("/users/123");

    expect(res.status).toBe(404);
  });
});
```

**13.Run the test**

Npm test

For Redis

🧪 **app.test.ts**

```
import { vi } from "vitest";

// ☑ mock redis BEFORE import
vi.mock("./redisClient", () => {
  return {
    redisClient: {
      get: vi.fn(),
      set: vi.fn(),
    },
  };
});

import request from "supertest";
import { describe, it, expect } from "vitest";
import app from "./app";
import { redisClient } from "./redisClient";

describe("GET /user/:id", () => {
  it("should return from cache", async () => {
    (redisClient.get as any).mockResolvedValue(
      JSON.stringify({ id: "1", name: "Prajwal" })
    );

    const res = await request(app).get("/user/1");

    expect(res.body.source).toBe("cache");
    expect(res.body.data.name).toBe("Prajwal");
  });

  it("should return from db and set cache", async () => {
    (redisClient.get as any).mockResolvedValue(null);

    const res = await request(app).get("/user/1");
```

```
    expect(res.body.source).toBe("db");
    expect(redisClient.set).toHaveBeenCalled();
  });
});
```

## Deepmock

### 🧠 What is Deep Mock?

**Problem:**

**Normal mock:**

```
vi.mock("./prisma", () => ({
  prisma: {}
}));
```

**👉 This fails because:**

**prisma.user.findUnique(...)**

**→ user is undefined**

**Solution: Deep Mock**

**A deep mock automatically mocks nested properties**

**prisma.user.findUnique**
**prisma.post.create**
**prisma.order.update**

**👉 All exist + are mock functions**

### 🔥 Library for this

**Use:**

```
npm install -D vitest-mock-extended
```

### 🚀 Step-by-Step (Deep Mock with Prisma)

---

### 🧱 1. Setup Mock File

**Create:**

src/tests/mocks/prisma.mock.ts

```
import { mockDeep, mockReset } from "vitest-mock-extended";
import type { PrismaClient } from "../../generated/prisma";

// create deep mock
export const prismaMock = mockDeep<PrismaClient>();

// reset before each test
beforeEach(() => {
  mockReset(prismaMock);
});
```

### ⚙️ 2. Mock Prisma Module

**In your test file:**

```
import { vi } from "vitest";
import { prismaMock } from "./mocks/prisma.mock";

// replace real prisma
vi.mock("../prisma/client", () => ({
  prisma: prismaMock,
}));
```

### 🧪 3. Use in Test

```
import request from "supertest";
import { describe, it, expect } from "vitest";
import app from "../app";
import { prismaMock } from "./mocks/prisma.mock";

describe("GET /users/:id", () => {
  it("should return user", async () => {
    prismaMock.user.findUnique.mockResolvedValue({
      id: "123",
      name: "Prajwal",
      email: "test@test.com",
```

```
  });

  const res = await request(app).get("/users/123");

  expect(res.status).toBe(200);
  expect(res.body.name).toBe("Prajwal");
 });
});
```

## Mock return value

**Mock Return Values (Vitest)**

**This is about:**

**“When this function is called → what should it return?”**

---

✅ **Basic Mock Return**

```
const fn = vi.fn();

fn.mockReturnValue("hello");

expect(fn()).toBe("hello");
```

👉 **Always returns "hello"**

## Mocks vs Spies (VERY IMPORTANT)

**This is where most devs get confused.**

🧪 **Mock = Fake Function**

```
const fn = vi.fn();

fn.mockReturnValue(10);
```

👉 **You create a fake function**

🕵️ Spy = Watch Real Function

```
const obj = {
  add: (a, b) => a + b,
};

const spy = vi.spyOn(obj, "add");

obj.add(1, 2);

expect(spy).toHaveBeenCalled();
```

👉 You track usage of real function

Spy Example

```
import * as service from "../services/user.service";

const spy = vi.spyOn(service, "getUserById");

await request(app).get("/users/123");

expect(spy).toHaveBeenCalledWith("123");
```

👉 You are checking:

- Was service called correctly?

🧠 Final Mental Model

Mock → Replace dependency

Spy → Observe behavior